

1 . Build a Serverless REST API using AWS

Design and implement a serverless backend REST API using AWS services. The API should allow basic user management operations such as creating a user, retrieving all users, and retrieving a specific user by ID.

Architecture Requirements

Your solution must follow a serverless architecture using the following AWS services:

- API Gateway (HTTP API) — to expose REST endpoint
- AWS Lambda — to implement application logic
- Amazon DynamoDB — to store user data
- IAM Role — to allow Lambda to access DynamoDB

The request flow should be:

Client → API Gateway → Lambda → DynamoDB → Response

Ans:

Tables (1) Info

Last updated March 10, 2026, 16:17 (UTC+5:30)

Actions

Delete

Create table

Find tables

Filter by tag Any tag key

Filter by tag value Any tag value

< 1 >

☐	Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection	Favorite	Read capacity mode	Write capacity mode	Total size	Table class
☐	Users	Active	Id (S)	-	0	0	Off	☆	On-demand	On-demand	0 bytes	Standard

LambdaExecutionRole Info

Delete

Allows Lambda functions to call AWS services on your behalf.

Summary

Edit

Creation date

March 10, 2026, 16:29 (UTC+05:30)

Last activity

-

ARN

arn:aws:iam::596059883112:role/LambdaExecutionRole

Maximum session duration

1 hour

Permissions

Trust relationships

Tags

Last Accessed

Revoke sessions

Permissions policies (2) Info

Simulate

Remove

Add permissions

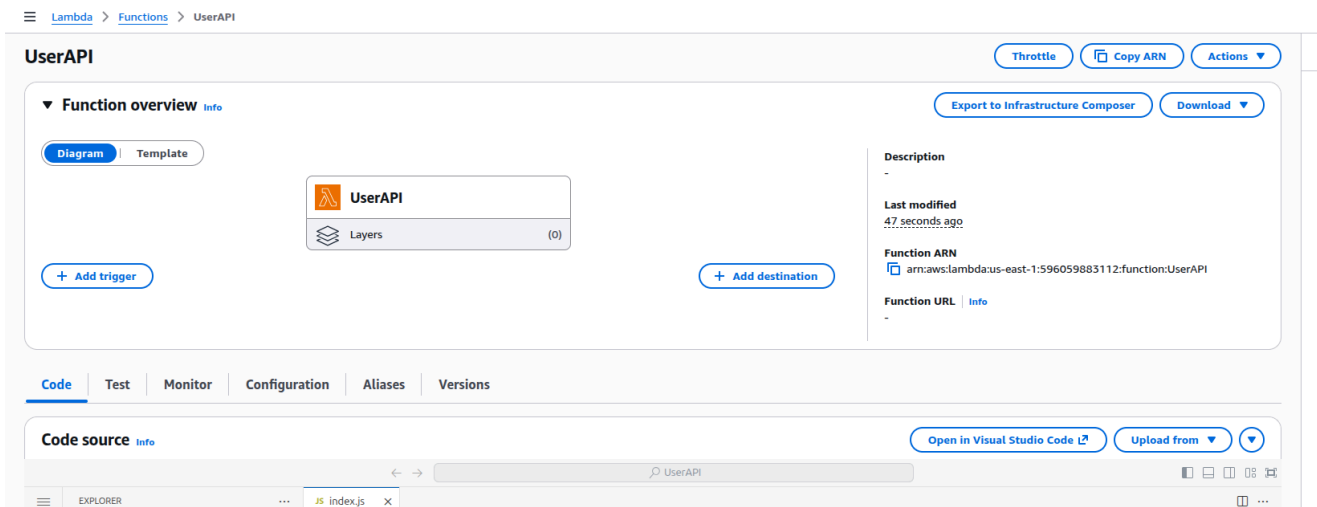
You can attach up to 10 managed policies.

Search

Filter by Type All types

< 1 >

☐	Policy name	Type	Attached entities
☐	AmazonDynamoDBFullAccess	AWS managed	1
☐	AWSLambdaDynamoDBExecutionRole	AWS managed	1



Lambda Code :

```
const { DynamoDBClient } = require("@aws-sdk/client-dynamodb");
const { DynamoDBDocumentClient, PutCommand, ScanCommand, GetCommand } =
require("@aws-sdk/lib-dynamodb");
```

```
const client = new DynamoDBClient({});
const dynamo = DynamoDBDocumentClient.from(client);
```

```
const TABLE_NAME = "Users";
```

```
exports.handler = async (event) => {
```

```
    const method = event.requestContext.http.method;
    const path = event.rawPath;

    try {
        if (method === "POST" && path === "/users") {

            const body = JSON.parse(event.body);

            const params = {
                TableName: TABLE_NAME,
                Item: {
                    id: body.id,
                    name: body.name,
                    email: body.email
                }
            };

            await dynamo.put(params).promise();

            return {
                statusCode: 201,
                body: JSON.stringify({ message: "User created successfully" })
            };
        }
    }
}
```

```

    if (method === "GET" && path === "/users") {

        const params = {
            TableName: TABLE_NAME
        };

        const data = await dynamo.scan(params).promise();

        return {
            statusCode: 200,
            body: JSON.stringify(data.Items)
        };
    }

    if (method === "GET" && event.pathParameters) {

        const id = event.pathParameters.id;

        const params = {
            TableName: TABLE_NAME,
            Key: { id }
        };

        const data = await dynamo.get(params).promise();

        return {
            statusCode: 200,
            body: JSON.stringify(data.Item)
        };

        return {
            statusCode: 400,
            body: JSON.stringify({ message: "Invalid request" })
        };
    } catch (error) {

        return {
            statusCode: 500,
            body: JSON.stringify(error)
        };
    }
}

```

```

};

```

Review and create

API name and integrations

API name

MyAPIs

IP address type

IPv4

Integrations

- UserAPI (Lambda)

Routes

Routes

- GET /users → UserAPI (Lambda)
- POST /users → UserAPI (Lambda)
- GET /users/{id} → UserAPI (Lambda)

Stages

Stages

- \$default (Auto-deploy: enabled)

Cancel

Previous

Create

Routes

Stage: -

Deploy

Routes for MyAPIs

Create

Search

▼ /users

GET

POST

▼ /{id}

GET

Route details

Delete

Edit

GET /users (ID: fjar7af)

ARN

arn:aws:apigateway:us-east-1::apis/n96xycthxj/routes/fjar7af

Authorization

Authorizers protect your API against unauthorized requests. Routes with no authorization attached are open.

No authorizer attached to this route.

Attach authorization

Integration

The integration is the backend resource that this route calls when it receives a request.

mpdk3xj

Configure

Routes

Stage: -

Deploy

Routes for MyAPIs

Create

Search

▼ /users

GET

POST

▼ /{id}

GET

Route details

Delete

Edit

POST /users (ID: jh3jj85)

ARN

arn:aws:apigateway:us-east-1::apis/n96xycthxj/routes/jh3jj85

Authorization

Authorizers protect your API against unauthorized requests. Routes with no authorization attached are open.

No authorizer attached to this route.

Attach authorization

Integration

The integration is the backend resource that this route calls when it receives a request.

s3l7jwu

Configure

Routes

Stage: -

Deploy

Routes for MyAPIs

Create

Search

▼ /users

GET

POST

▼ /{id}

GET

Route details

Delete

Edit

GET /users/{id} (ID: 3wu5os6)

ARN

arn:aws:apigateway:us-east-1::apis/n96xycthxj/routes/3wu5os6

Authorization

Authorizers protect your API against unauthorized requests. Routes with no authorization attached are open.

No authorizer attached to this route.

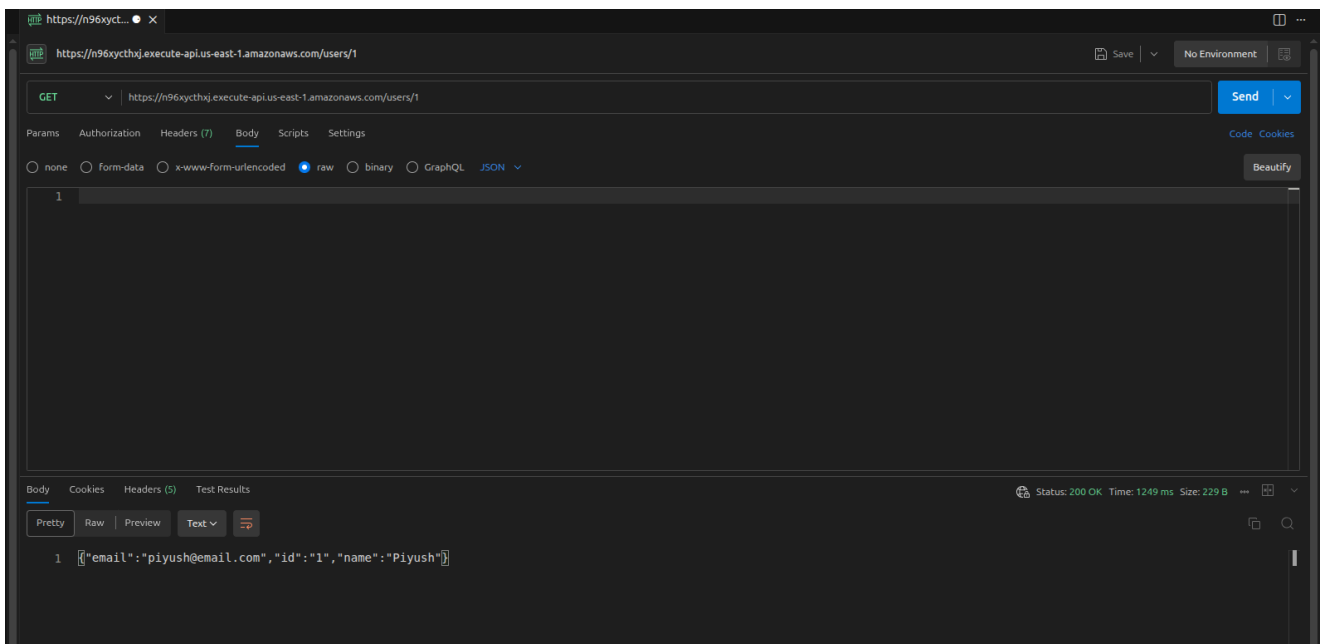
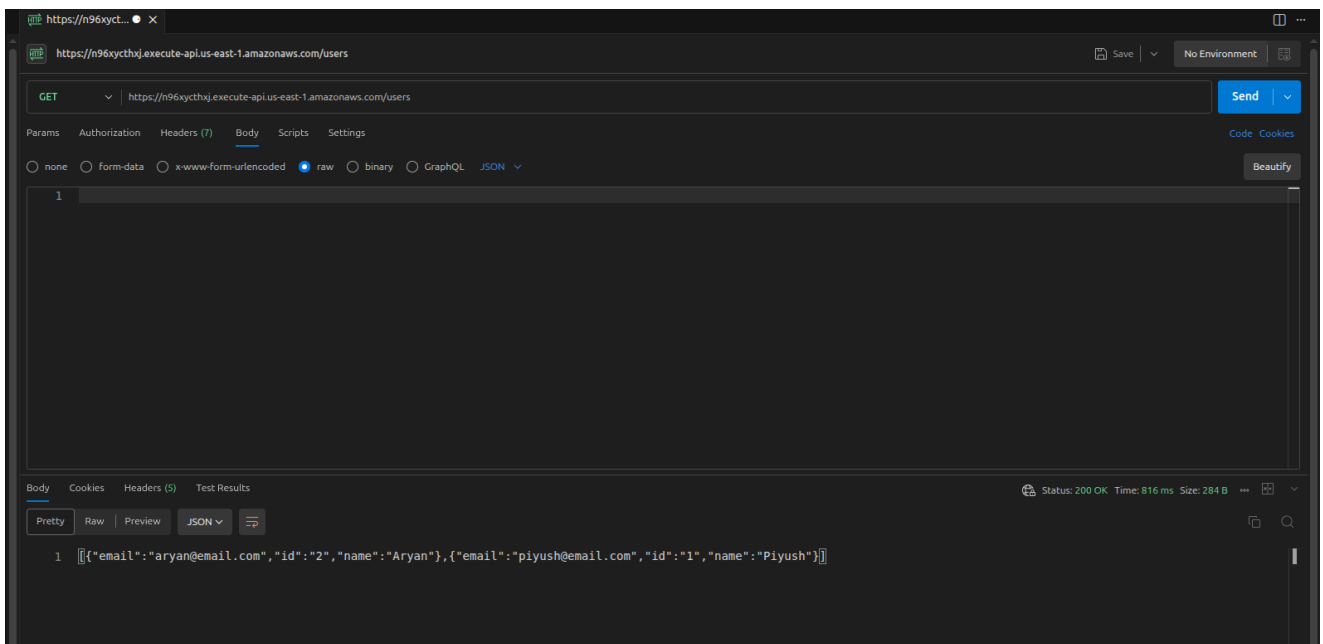
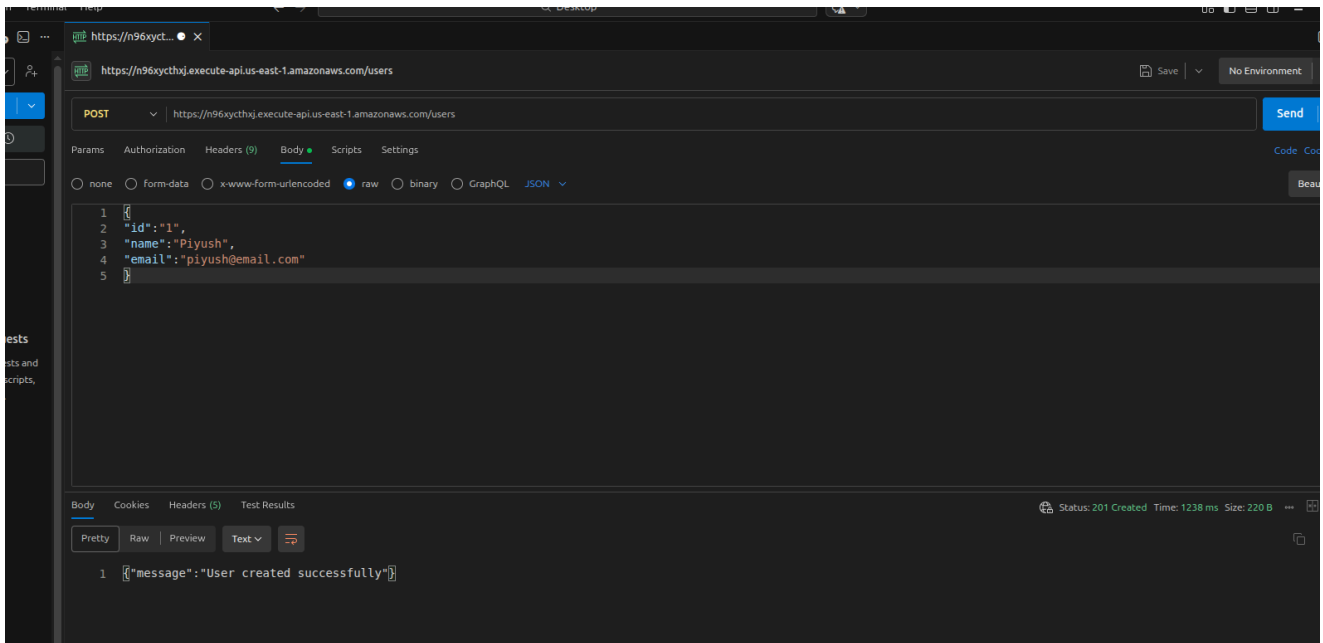
Attach authorization

Integration

The integration is the backend resource that this route calls when it receives a request.

q8hrzm

Configure



In this task, I built a **serverless REST API for basic user management** using AWS services. I used **API Gateway (HTTP API)** to expose REST endpoints and **AWS Lambda** to handle the application logic. I created a **DynamoDB table** to store user information such as user ID and user details. The Lambda function interacts with DynamoDB to **create a new user, retrieve all users, and retrieve a specific user by ID**. I also configured an **IAM role** to allow Lambda to access DynamoDB securely. The request flow in this system is **Client → API Gateway → Lambda → DynamoDB → Response**.

2. Secure a Serverless REST API using API Gateway and Lambda

Build a serverless REST API using AWS API Gateway and AWS Lambda and secure the API using an API Key and Usage Plan so that only authorized requests can access the endpoint.

Architecture

Your implementation must follow this architecture:

Client → API Gateway (REST API) → Lambda → Response

The API must reject requests that do not include a valid API key.

Ans:

The screenshot displays the AWS API Gateway console. The top section shows a list of APIs with one API named 'MyRESTApi' in an 'Available' state. Below this, the 'Resources' section is visible, showing the hierarchy: '/' > '/users'. The '/users' resource has two methods: 'GET' (selected) and 'POST'. The 'GET' method is configured with a 'Method request' and 'Integration request' flow. The 'Method request settings' for the 'GET' method are shown, including 'Authorization: NONE', 'Request validator: None', 'API key required: True', and 'SDK operation name: Generated based on method and path'. The 'Request paths' section is empty, indicating no request paths are defined for this method.

Usage plans (1/1) Info

Last updated

March 10, 2026, 20:58 (UTC+05:30)

⌄

Edit

Delete

Create usage plan

	Name	ID	Request rate	Burst	Quota
☐	UserAPIUsagePlan	1sgm2y	10 per second	20	100 per month

UserAPIUsagePlan

Last updated

March 10, 2026, 21:00 (UTC+05:30)

⌄

Actions

Export usage data

Usage plan details

Usage plan ID

1sgm2y

Description

-

AWS Marketplace product code

-

Rate

10 requests per second

Burst

20 requests

Quota

100 requests per month

Associated stages

Associated API keys

Tags

Associated stages (1) Info

Edit

Remove

Add stage

	API	Stage	Method throttling
☐	MyRESTapi	dev	-

Now Testing without API Key

https://jqap24c...

https://jqap24cd5c.execute-api.us-east-1.amazonaws.com/dev/users

Save

No Environment

GET

https://jqap24cd5c.execute-api.us-east-1.amazonaws.com/dev/users

Send

Params

Authorization

Headers (7)

Body

Scripts

Settings

Type

API Key

The authorization header will be automatically generated when you send the request. Learn more about [authorization](#).

Key

Value

Add To

Key

Value

Headers

Body

Cookies

Headers (7)

Test Results

Status: 403 Forbidden

Time: 776 ms

Size: 290 B

Pretty

Raw

Preview

JSON

1

{ "message": "Forbidden" }

https://jqap24c...

https://jqap24cd5c.execute-api.us-east-1.amazonaws.com/dev/users

Save

No Environment

POST

https://jqap24cd5c.execute-api.us-east-1.amazonaws.com/dev/users

Send

Params

Authorization

Headers (9)

Body

Scripts

Settings

☐ none
☐ form-data
☐ x-www-form-urlencoded
☒ raw
☐ binary
☐ GraphQL

JSON

1

{

2 "id": "3",

3 "name": "Rahul",

4 "email": "rahul@email.com"

5 }

Body

Cookies

Headers (7)

Test Results

Status: 403 Forbidden

Time: 751 ms

Size: 290 B

Pretty

Raw

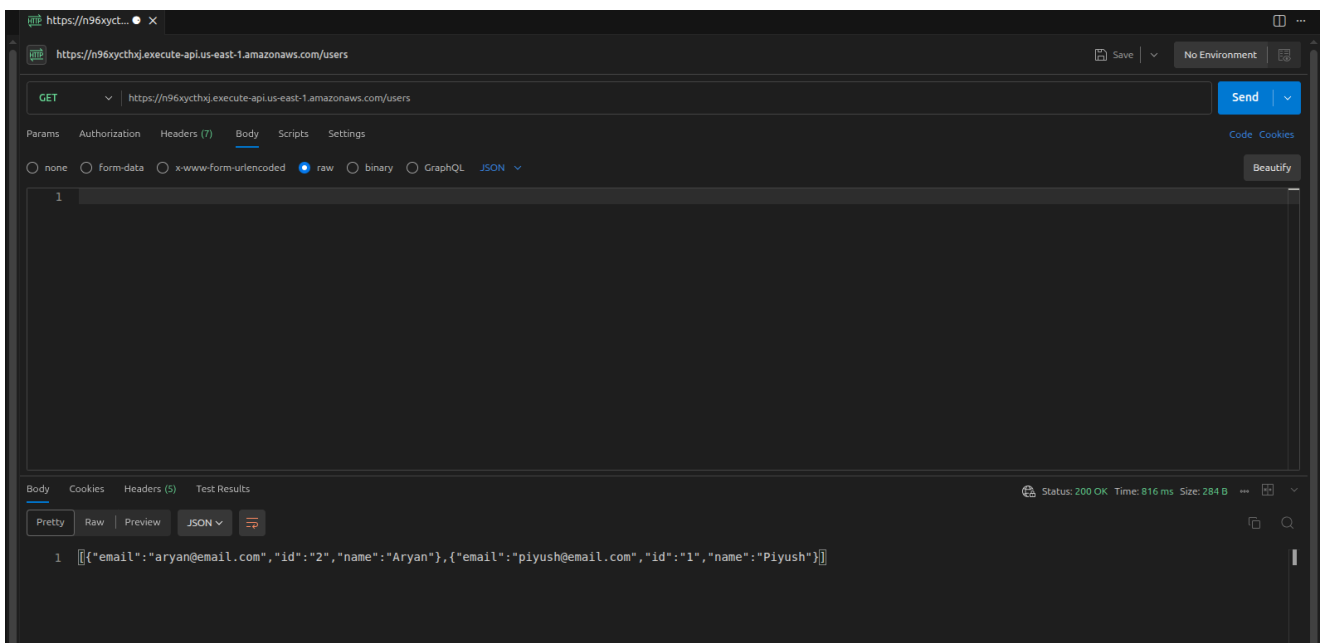
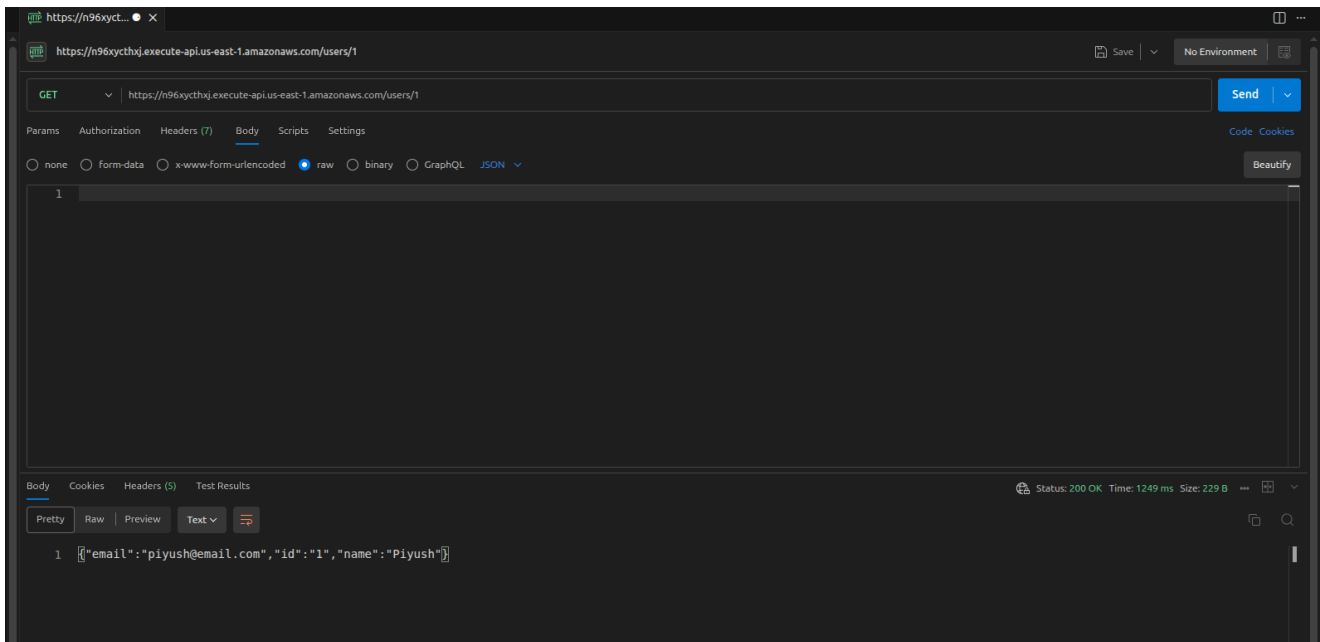
Preview

JSON

1

{ "message": "Forbidden" }

Now Testing with API Key



In this task, I built a **serverless REST API using API Gateway and Lambda** and secured it using an **API Key and Usage Plan**. API Gateway exposes the endpoint, and Lambda handles the backend logic. I created an **API key** and attached it to a **usage plan** so that only authorized clients can access the API. If a request is sent without a valid API key, API Gateway automatically rejects the request. This ensures that the API is protected and only allowed users can access the endpoint. The request flow is **Client → API Gateway → Lambda → Response**.

3. Implement Request Validation in API Gateway using a Required Query Parameter.

Build a serverless REST API using AWS API Gateway and AWS Lambda where API Gateway validates incoming requests before invoking Lambda.

The API must require a query parameter, and if the parameter is missing, the request

must be rejected with 400 Bad Request without executing the Lambda function.

Architecture

Your solution must follow this architecture:

Client → API Gateway (Request Validation) → Lambda → Response

The API Gateway should perform validation before the request reaches the Lambda function

Ans:

The screenshot displays the AWS Lambda console for the **queryValidatorDemo** function. The **Function overview** tab is active, showing a diagram of the function's architecture. The function is triggered by an **API Gateway**. The function's details include:

- Description:** -
- Last modified:** 5 minutes ago
- Function ARN:** `arn:aws:lambda:us-east-1:596059883112:function:queryValidatorDemo`
- Function URL:** [Info](#)

The **API Gateway** stage is configured with the following details:

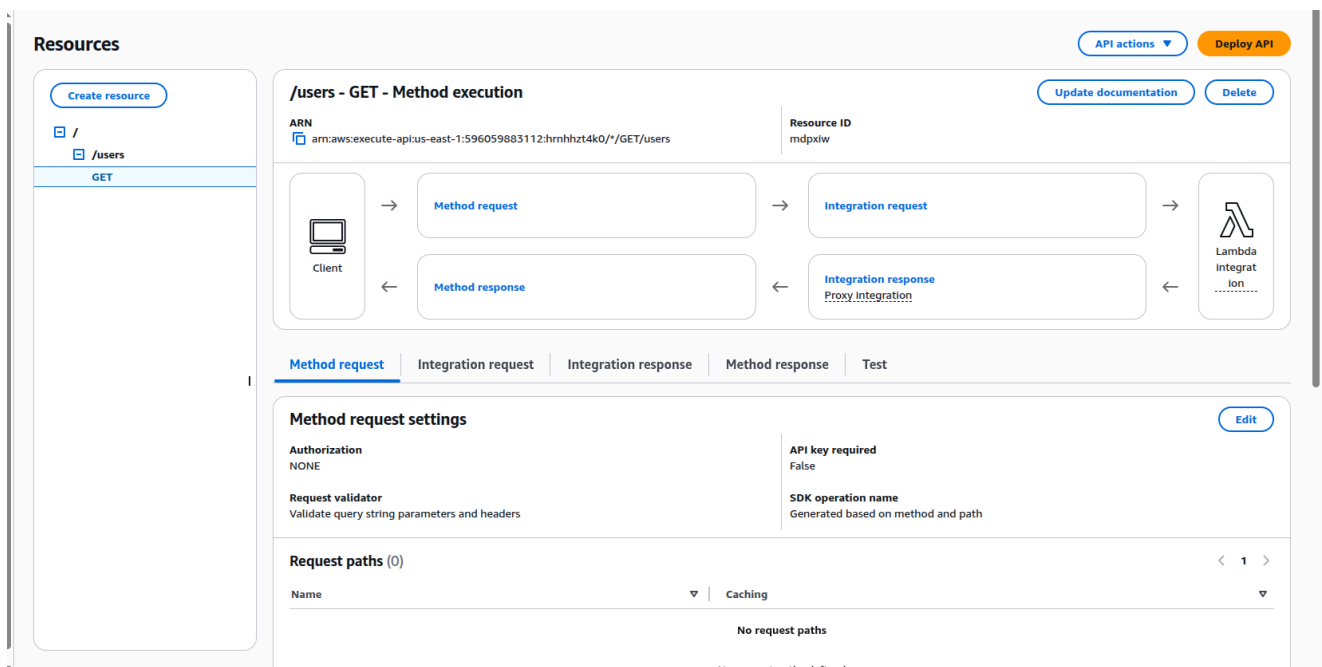
- Stage name:** dev
- Cache cluster:** Inactive
- Default method-level caching:** Inactive
- Invoke URL:** <https://hrmhz4k0.execute-api.us-east-1.amazonaws.com/dev>
- Active deployment:** brt3yq on March 11, 2026, 16:46 (UTC+05:30)

The **Logs and tracing** section shows the following settings:

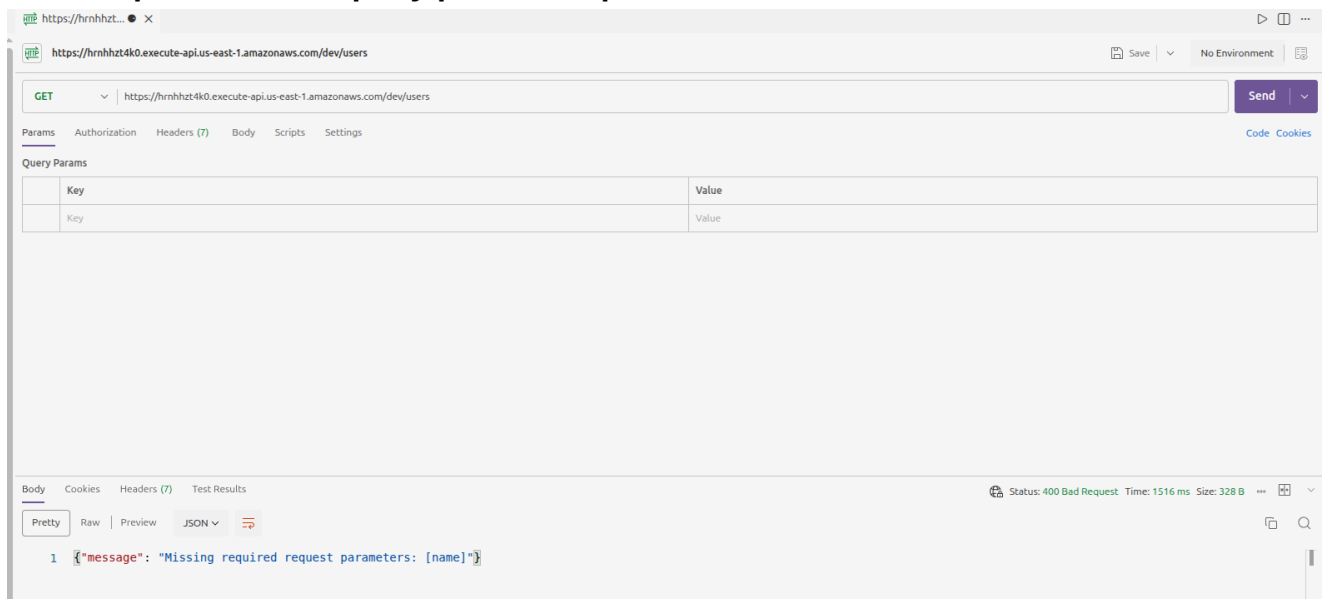
- CloudWatch logs:** Inactive
- Detailed metrics:** Inactive
- X-Ray tracing:** Inactive
- Data tracing:** Inactive
- Custom access logging:** Inactive

The **Stage variables** tab is selected, showing the following variables:

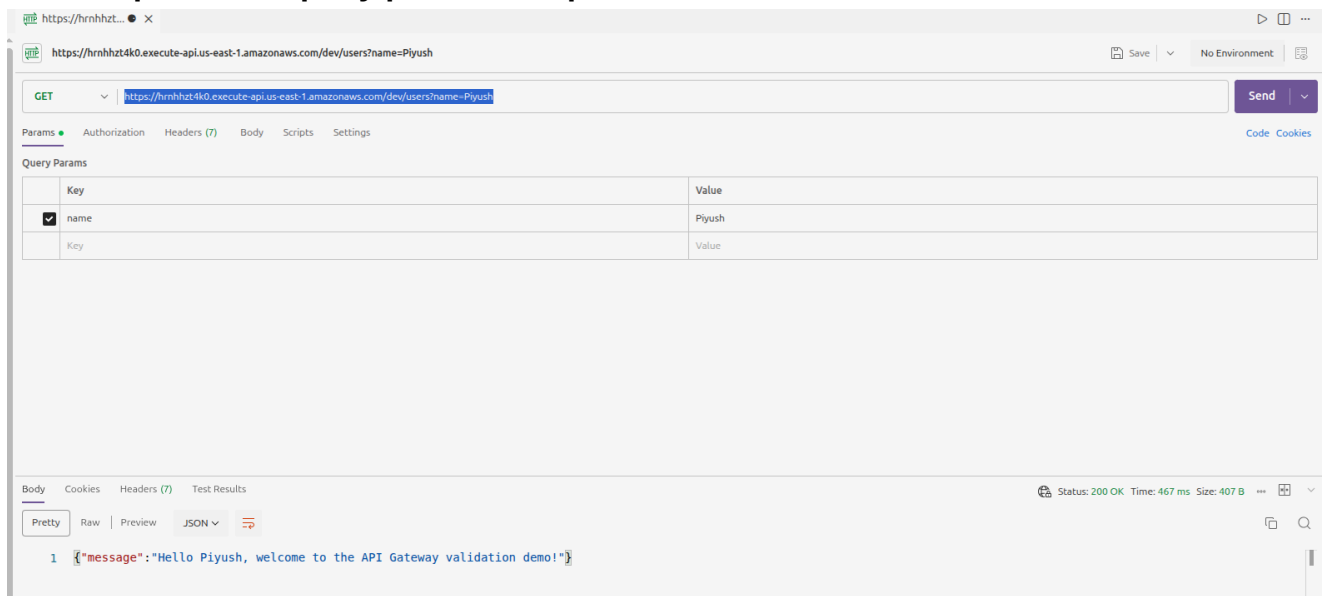
- Stage variables:** -
- Deployment history:** -
- Documentation history:** -
- Canary:** -
- Tags:** -



GET Request with No query parameter passed



GET Request with query parameters passed.



In this task, I built a serverless REST API using **AWS API Gateway and Lambda (Node.js 20)**. I configured API Gateway to require a **query parameter called name**. I enabled **request validation** so that if the query parameter is missing, API Gateway automatically returns a **400 Bad Request** and the Lambda function is not executed. This ensures that invalid requests are rejected before reaching the backend.

4. Build a Production-Style Asynchronous Order Processing System using AWS Serverless Services

Design and implement a production-style asynchronous order processing system using AWS serverless services.

The system must accept orders through an API, place them in a queue, and process them asynchronously using a worker Lambda function.

Architecture

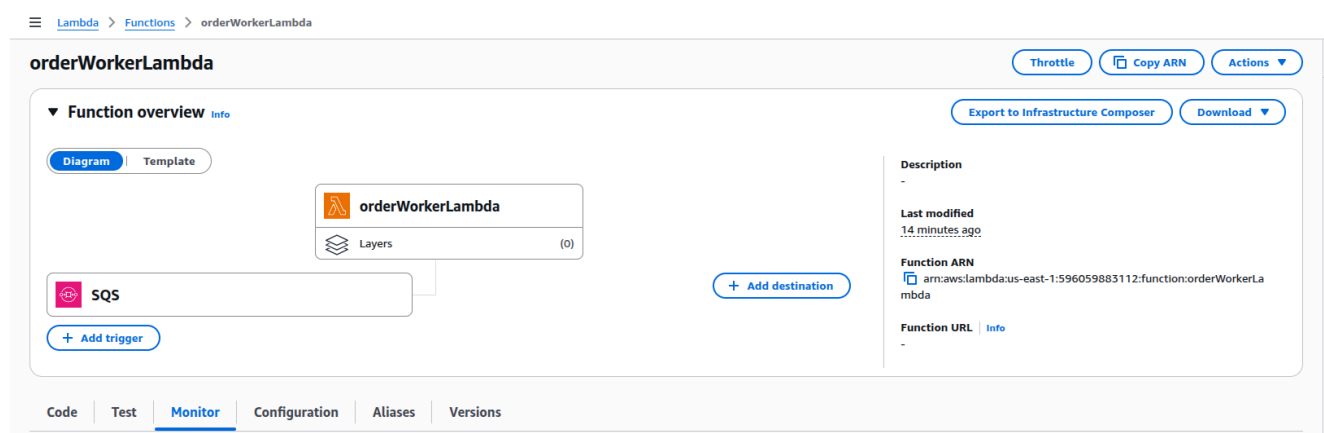
Your solution must implement the following architecture:

Client → API Gateway → Producer Lambda → SQS Queue → Worker Lambda → CloudWatch Logs

Explanation:

1. The client sends an order request.
2. API Gateway receives the request.
3. The Producer Lambda validates the request and sends the order to an SQS queue.
4. The client receives an immediate response.
5. The Worker Lambda is triggered by the SQS queue and processes the order asynchronously.

Ans:



Lambda > Functions > orderProducerLamba

orderProducerLamba

ThrottleCopy ARNActions

Function overviewInfo

DiagramTemplate

orderProducerLamba

Layers(0)

API Gateway

Add trigger

Add destination

Description-

Last modified9 minutes ago

Function ARNarn:aws:lambda:us-east-1:596059883112:function:orderProducerLamba

Function URLInfo

CodeTestMonitorConfigurationAliasesVersions

Resources

Create resource

/orders

POST

API actionsDeploy API

Update documentationDelete

/orders - POST - Method execution

ARNarn:aws:execute-api:us-east-1:596059883112:mhbfgytpz1/*/*POST/orders

Resource ID4x1737

Client

Method request

Integration request

Method response

Integration responseProxy integration

Lambda Integration

Method requestIntegration requestIntegration responseMethod responseTest

Method request settings

Edit

AuthorizationNONE

API key requiredFalse

Request validatorNone

SDK operation nameGenerated based on method and path

Request paths(0)

<1>

Name

Caching

No request paths

order-processing-queue

EditDeletePurgeSend and receive messagesStart DLQ redrive

DetailsInfo

Nameorder-processing-queue

TypeStandard

ARNarn:aws:sqs:us-east-1:596059883112:order-processing-queue

EncryptionAmazon SQS key (SSE-SQS)

URLhttps://sqs.us-east-1.amazonaws.com/596059883112/order-processing-queue

Dead-letter queue-

More

Queue policiesMonitoringSNS subscriptionsLambda triggersEventBridge PipesDead-letter queueTaggingEncryptionDead-letter queue redrive tasks

Access policyInfo

Edit

Define who can access your queue.

```
{  "Version": "2012-10-17",  "Id": "__default_policy_ID",  "Statement": [    {      "Sid": "__owner_statement",      "Effect": "Allow",      "Principal": {        "AWS": "arn:aws:iam:596059883112:root"      },      "Action": "SQS:*",      "Resource": "arn:aws:sqs:us-east-1:596059883112:order-processing-queue"    }  ]}
```

https://mhbfgvt... X

https://mhbfgvtzp1.execute-api.us-east-1.amazonaws.com/prod/orders

Save | No Environment

POST | https://mhbfgvtzp1.execute-api.us-east-1.amazonaws.com/prod/orders

Send

Params | Authorization | Headers (9) | Body | Scripts | Settings

Code | Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | JSON

Beautify

```
1 {
2   "customerName": "Piyush",
3   "productId": "P1001",
4   "quantity": 2,
5   "price": 500
6 }
```

Body | Cookies | Headers (7) | Test Results

Status: 200 OK Time: 4.54 s Size: 462 B

Pretty | Raw | Preview | JSON

```
1 { "message": "Order received and queued for processing", "orderId": "98a69ad4-6ef0-4b7d-8a82-f047a6bce381", "status": "PROCESSING" }
```

Log events

Actions | Start tailing | Create metric filter

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#)

Filter events - press enter to search

Clear | 1m | 30m | 1h | 12h | Custom | UTC timezone | Display

Timestamp	Message
No older events at this moment. Retry	
2026-03-11T11:56:34.438Z	INIT_START Runtime Version: nodejs:20.v95 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:2d788b26ac416854c5eb831c9f46cb2e0a0ddcee84b1a9b114acbdfeb644c8c1
2026-03-11T11:56:34.857Z	START RequestId: 137cf5af-8cff-4623-8883-3f068fd89bfe Version: \$LATEST
2026-03-11T11:56:34.859Z	2026-03-11T11:56:34.859Z 137cf5af-8cff-4623-8883-3f068fd89bfe INFO Incoming Event: {"resource":"/orders","path":"/orders","httpMethod":"POST","headers":{"Accept":"","Accept-...}}
2026-03-11T11:56:34.884Z	2026-03-11T11:56:34.884Z 137cf5af-8cff-4623-8883-3f068fd89bfe INFO Order Payload: { orderId: '98a69ad4-6ef0-4b7d-8a82-f047a6bce381', customerName: 'Piyush', productId: 'P1001'...
2026-03-11T11:56:35.825Z	END RequestId: 137cf5af-8cff-4623-8883-3f068fd89bfe
2026-03-11T11:56:35.825Z	REPORT RequestId: 137cf5af-8cff-4623-8883-3f068fd89bfe Duration: 967.85 ms Billed Duration: 1384 ms Memory Size: 128 MB Max Memory Used: 92 MB Init Duration: 415.24 ms
No newer events at this moment. Auto retry paused . Resume	

Queues (1)

Edit | Delete | Send and receive messages | Actions | Create queue

Search queues by prefix

< 1 >

Name	Type	Created	Messages available	Messages in flight	Encryption	Content-based deduplication
order-processing-queue	Standard	2026-03-11T17:09+05:30	0	1	Amazon SQS key (SSE-SQS)	-

Log events

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#)

Q

Filter events - press enter to search

Clear

1m

30m

1h

12h

Custom

UTC timezone

Display

▶

Timestamp

Message

No older events at this moment. [Retry](#)

▶

2026-03-11T12:03:05.229Z

INIT_START Runtime Version: nodejs:20.v95 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:2d788b26ac416854c5eb831c9f46cb2e0a8ddcee84b1a9b114acbdfeb644c8c1

▶

2026-03-11T12:03:05.373Z

START RequestId: 428fb4ef-f3a1-550a-ba02-d6c4af959c08 Version: \$LATEST

▶

2026-03-11T12:03:05.387Z

2026-03-11T12:03:05.387Z 428fb4ef-f3a1-550a-ba02-d6c4af959c08 INFO SQS Event Received: { "Records": [{ "messageId": "974902a8-43ba-4508-be53-5fb495bb30f3", "receiptHandle": "...

▶

2026-03-11T12:03:05.409Z

2026-03-11T12:03:05.409Z 428fb4ef-f3a1-550a-ba02-d6c4af959c08 INFO Processing Order ID: 30beb17b-9f27-4319-bb2f-319a3f34eaf0

▶

2026-03-11T12:03:05.426Z

2026-03-11T12:03:05.426Z 428fb4ef-f3a1-550a-ba02-d6c4af959c08 INFO Processing payment for order: 30beb17b-9f27-4319-bb2f-319a3f34eaf0

▶

2026-03-11T12:03:05.427Z

2026-03-11T12:03:05.427Z 428fb4ef-f3a1-550a-ba02-d6c4af959c08 INFO Packing order: 30beb17b-9f27-4319-bb2f-319a3f34eaf0

▶

2026-03-11T12:03:07.429Z

2026-03-11T12:03:07.429Z 428fb4ef-f3a1-550a-ba02-d6c4af959c08 INFO Shipping order: 30beb17b-9f27-4319-bb2f-319a3f34eaf0

▶

2026-03-11T12:03:07.429Z

2026-03-11T12:03:07.429Z 428fb4ef-f3a1-550a-ba02-d6c4af959c08 INFO Order processed successfully: 30beb17b-9f27-4319-bb2f-319a3f34eaf0

▶

2026-03-11T12:03:07.432Z

END RequestId: 428fb4ef-f3a1-550a-ba02-d6c4af959c08

▶

2026-03-11T12:03:07.433Z

REPORT RequestId: 428fb4ef-f3a1-550a-ba02-d6c4af959c08 Duration: 2059.21 ms Billed Duration: 2290 ms Memory Size: 128 MB Max Memory Used: 69 MB Init Duration: 140.47 ms

No newer events at this moment. [Auto retry paused](#). [Resume](#)

In this task, I designed an **asynchronous order processing system** using **API Gateway**, **Lambda**, and **Amazon SQS**. The client sends an order request to API Gateway, which triggers a **Producer Lambda** function. The Lambda validates the order and sends it to an **SQS queue**. A **Worker Lambda** is then triggered by SQS to process the order asynchronously, allowing the system to handle requests efficiently and process orders in the background.

5. Build an Order Processing Workflow Using AWS Step Functions

Design and implement a serverless order processing system using AWS Step Functions.

The system should simulate how an e-commerce platform processes an order after a customer submits it.

Requirements

Create a Step Functions workflow that performs the following steps:

1. Validate an incoming order request.
2. Process the payment for the order.
3. Reserve inventory for the ordered product.
- 4.. Save the order details into a DynamoDB table.
5. Send a notification to the customer.

Each step must be implemented as a separate Lambda function.

Ans:

Tables (1)

Filter by tag

Any tag key

Filter by tag value

Any tag value

Find tables

1

OrdersTable

OrdersTable

Settings

Indexes

Monitor

Global tables

Backups

Exports and streams

Permissions

Protect your DynamoDB table from accidental writes and deletes

When you turn on point-in-time recovery (PITR), DynamoDB backs up your table data automatically so that you can restore to any given second in the preceding 1 to 35 days. Additional charges apply. [Learn more](#)

Edit PITR

General information

Partition key

orderid (String)

Sort key

-

Capacity mode

On-demand

Alarms

No active alarms

Point-in-time recovery (PITR)

Off

Item count

0

Average item size

0 bytes

Resource-based policy

Not active

Table status

Active

Table size

0 bytes

Amazon Resource Name (ARN)

arn:aws:dynamodb:us-east-1:596059883112:table/OrdersTable

Get live item count

Additional info

Read/write capacity

The read/write capacity mode controls how you are charged for read and write throughput and how you manage capacity.

Capacity mode

On-demand

Maximum read request units

-

Maximum write request units

-

Edit

Lambda > Functions > validateOrderLambda

validateOrderLambda

Throttle

Copy ARN

Actions

Function overview

Diagram

Template

validateOrderLambda

Layers (0)

+ Add trigger

+ Add destination

Description

-

Last modified

2 minutes ago

Function ARN

arn:aws:lambda:us-east-1:596059883112:function:validateOrderLambda

Function URL

Info

Lambda > Functions > processPaymentLambda

processPaymentLambda

Throttle

Copy ARN

Actions

Function overview

Diagram

Template

processPaymentLambda

Layers (0)

+ Add destination

Description

-

Last modified

2 minutes ago

Function ARN

arn:aws:lambda:us-east-1:596059883112:function:processPaymentLambda

Function URL

Info

Lambda > Functions > reserveInventoryLambda

reserveInventoryLambda

Throttle

Copy ARN

Actions

Function overview

Diagram

Template

reserveInventoryLambda

Layers (0)

+ Add destination

Description

-

Last modified

2 minutes ago

Function ARN

arn:aws:lambda:us-east-1:596059883112:function:reserveInventoryLambda

Function URL

Info

Lambda > Functions > saveOrderLambda

saveOrderLambda

Throttle

Copy ARN

Actions

Export to Infrastructure Composer

Download

Function overview

Diagram

Template

saveOrderLambda

Layers

(0)

+ Add destination

Description

-

Last modified

1 minute ago

Function ARN

arn:aws:lambda:us-east-1:596059883112:function:saveOrderLambda

Function URL

Info

Lambda > Functions > sendNotificationLambda

sendNotificationLambda

Throttle

Copy ARN

Actions

Export to Infrastructure Composer

Download

Function overview

Diagram

Template

sendNotificationLambda

Layers

(0)

+ Add destination

Description

-

Last modified

1 minute ago

Function ARN

arn:aws:lambda:us-east-1:596059883112:function:sendNotificationLambda

Function URL

Info

OrderProcessingWorkflow

Standard

Design

Code

Config

Exit

Actions

Execute

Save

Undo

Redo

Zoom in

Zoom out

Center

Duplicate

Delete

Feedback

Search

Actions

Flow

Patterns

Info

MOST POPULAR

AWS Lambda

Invoke

Amazon SNS

Publish

Amazon ECS

RunTask

AWS Step Functions

StartExecution

AWS Glue

StartJobRun

HTTPS APIS

HTTP Endpoint

Call HTTPS APIs

Updated

COMPUTE

Amazon Data Lifecycle Manager

Amazon EBS

Start

Lambda: Invoke

Validate Order

Lambda: Invoke

Process Payment

Lambda: Invoke

Reserve Inventory

Lambda: Invoke

Save Order

Lambda: Invoke

Send Notification

End

Workflow

Definition

The top level state machine properties for this workflow. [Learn more](#)

State machine query language

JSONPath

Start at

The state that is the starting point of the workflow.

Validate Order

Comment - optional

A human-readable description of the state machine.

Order Processing Workflow

TimeoutSeconds - optional

The maximum number of seconds an execution of the state machine can run. If it runs longer than the specified time, the execution fails with a States.Timeout.

600

Step Functions > State machines > OrderProcessingWorkflow > Execution: f527c017-2b32-4866-8ae7-1574b43317dd

Execution started successfully

Execution: f527c017-2b32-4866-8ae7-1574b43317dd

Edit state machine

New execution

Actions

Step Functions

Dashboard

State machines

Execution Inspector

Activities

Developer resources

Online learning workshop

Local Development

Data flow simulator

Feature spotlight

Documentation

Join our feedback panel

Free virtual workshops

Details

Execution input and output

Definition

Execution status

Succeeded

Execution type

Standard

Execution ARN

arn:aws:states:us-east-1:596059883112:execution:OrderProcessingWorkflow:f527c017-2b32-4866-8ae7-1574b43317dd

IAM role ARN

arn:aws:iam::596059883112:role/service-role/StepFunctions-OrderProcessingWorkflow-role-lwyetcdsd

State transitions

Learn more

7

Start time

Mar 11, 2026, 19:43:55.288 (UTC+05:30)

End time

Mar 11, 2026, 19:43:59.198 (UTC+05:30)

Duration

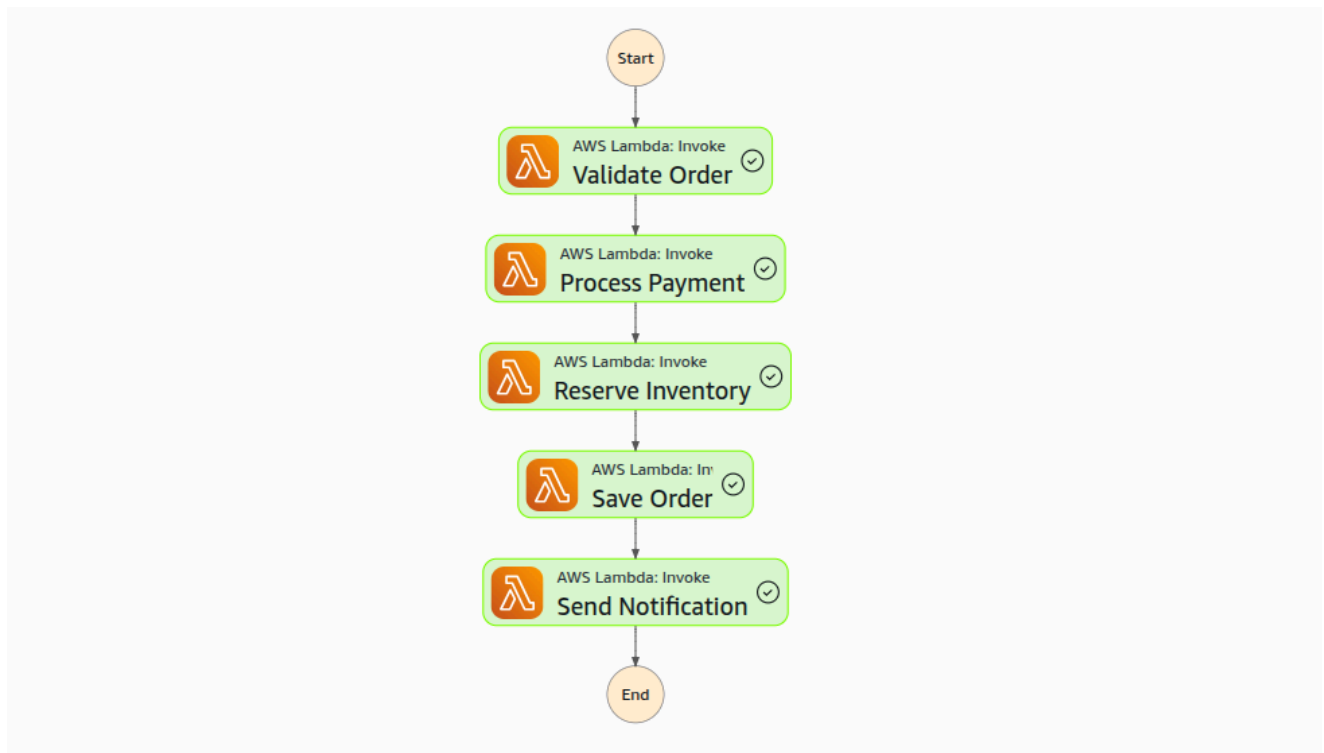
00:00:03.910

Alias

-

Version

-



In this task, I implemented an **order processing workflow** using **AWS Step Functions**. I created multiple Lambda functions for different steps such as **validating the order**, **processing payment**, **reserving inventory**, **saving the order in DynamoDB**, and **sending a notification to the customer**. Step Functions orchestrates these Lambda functions in sequence, simulating a real-world e-commerce order processing pipeline.

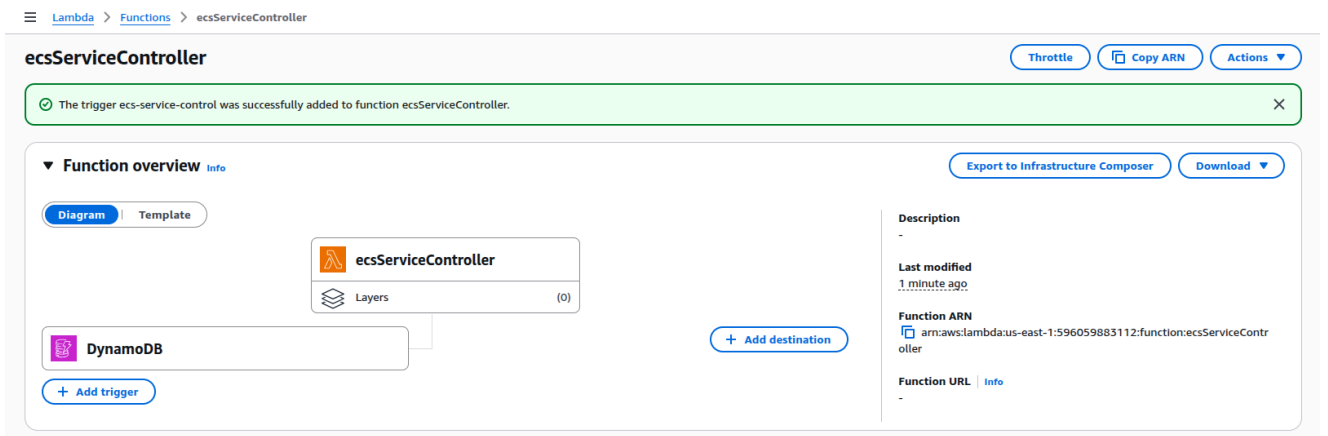
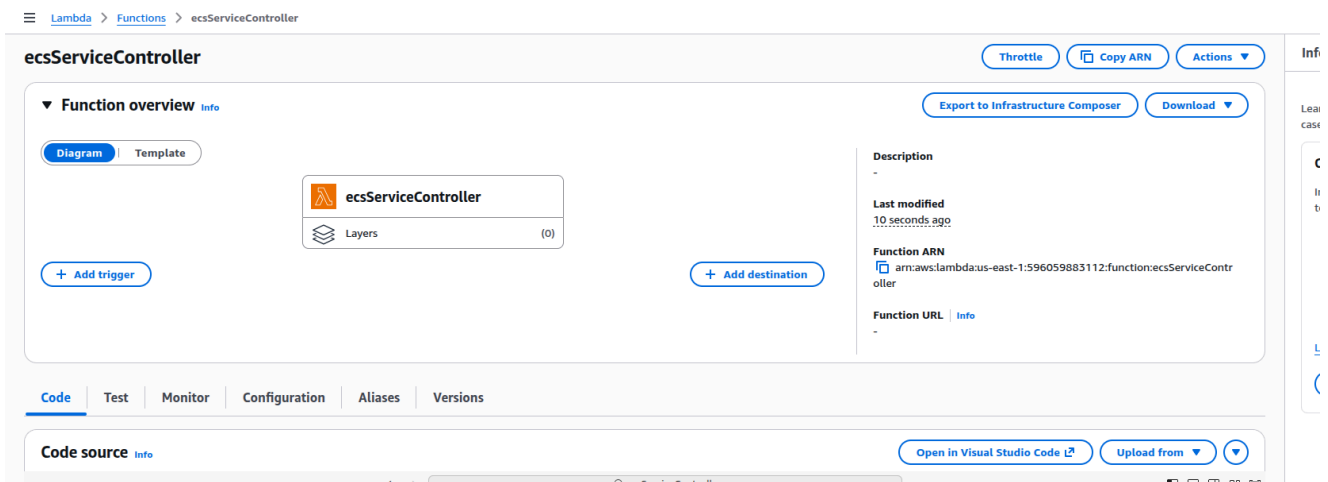
6. Build an event-driven automation system where a DynamoDB table stores ECS service name. When the value of a service changes from true to false, the corresponding ECS service should automatically stop running.

Ans:

The screenshot displays the AWS Management Console for the 'ecs-service-control' DynamoDB table. The interface includes a left-hand navigation pane with 'Tables (1)' and a search bar. The main content area shows the 'Settings' tab for the table. A prominent blue banner at the top indicates that 'Protect your DynamoDB table from accidental writes and deletes' is enabled, with a link to 'Learn more' and an 'Edit PITR' button. Below this, the 'General information' section provides details about the table's configuration:

- Partition key:** serviceName (String)
- Sort key:** -
- Capacity mode:** On-demand
- Table status:** Active
- Item count:** 0
- Table size:** 0 bytes
- Point-in-time recovery (PITR):** Off
- Resource-based policy:** Not active
- Amazon Resource Name (ARN):** arn:aws:dynamodb:us-east-1:596059883112:table/ecs-service-control

Additional options like 'Alarms', 'Average item size', and 'Additional info' are also visible.



In this task, I built an **event-driven automation system using DynamoDB Streams and Lambda**. I created a DynamoDB table to store ECS service names and their status. Whenever the value of a service changes from **true to false**, DynamoDB Streams triggers a Lambda function. The Lambda function then calls the **ECS UpdateService API** to set the desired count to zero, which automatically stops the ECS service.